

# Food App Backend API

---

License MIT

---

## Table of Contents

1. [Project Overview](#)
    - [1.1. Purpose & Core Functionality](#)
    - [1.2. Key Technologies Utilized](#)
    - [1.3. Database Schema & Models](#)
    - [1.4. API Design Principles](#)
  2. [Prerequisites](#)
    - [2.1. System Requirements](#)
    - [2.2. Software Installation & Verification](#)
  3. [Setup & Running the Application](#)
    - [3.1. Obtaining the Project Files](#)
    - [3.2. Project Directory Structure](#)
    - [3.3. Environment Variable Configuration](#)
    - [3.4. Building & Starting Docker Containers](#)
    - [3.5. Verifying Container Status](#)
    - [3.6. Database Initialization & Migrations](#)
    - [3.7. Populating Initial Data \(Optional\)](#)
  4. [API Endpoints Reference](#)
    - [Base URL](#)
    - [4.1. User Management\( \[/api/users\]\(#\) \)](#)
    - [4.2. Food Category Management\( \[/api/categories\]\(#\) \)](#)
    - [4.3. Food Item Management\( \[/api/foods\]\(#\) \)](#)
    - [4.4. Ingredient Management\( \[/api/ingredients\]\(#\) \)](#)
    - [4.5. Food Image Management\( \[/api/food\\\_images\]\(#\) \)](#)
  5. [Stopping the Application](#)
  6. [Troubleshooting Guide](#)
  7. [Contact Information](#)
- 

## 1. Project Overview

This document serves as a comprehensive guide for the **Food App Backend API** submission. This project demonstrates a robust, modular, and containerized backend solution for managing various food-related data entities.

## 1.1. Purpose & Core Functionality

The primary purpose of this API is to provide a programmatic interface for a food planning and tracking application. It enables full **CRUD (Create, Read, Update, Delete)** operations for essential data models, ensuring data integrity and efficient management of food-related information. This API can serve as the foundation for various frontend applications (web, mobile, desktop) that require interaction with food-related data.

## 1.2. Key Technologies Utilized

The backend is built upon a modern Python web stack, leveraging industry-standard tools for development, containerization, and data management:

- **Python 3.8+:** The core programming language.
- **Flask:** A microframework for building web applications and RESTful APIs in Python, known for its flexibility and simplicity.
- **Flask-RESTful:** An extension that provides tools for quickly building REST APIs with Flask, simplifying resource definitions.
- **Flask-SQLAlchemy:** A Flask extension that provides SQLAlchemy integration, acting as an Object Relational Mapper (ORM) to interact with databases using Python objects rather than raw SQL.
- **Flask-Migrate:** An extension based on Alembic, used for handling database schema migrations, ensuring smooth updates to the database structure as the application evolves.
- **Marshmallow:** A powerful library for object serialization/deserialization. It is used here for data validation, converting Python objects (SQLAlchemy models) to JSON for API responses, and converting incoming JSON request bodies into Python objects for database persistence.
- **Werkzeug Security:** Utilized for secure password hashing (SHA256) for user authentication, ensuring passwords are never stored in plain text.
- **PostgreSQL:** A highly stable, robust, and feature-rich open-source relational database system, chosen for its reliability and performance in managing structured data.
- **Docker:** A leading platform that enables developers to package applications and their dependencies into standardized units called containers. This ensures the application runs consistently across any environment.
- **Docker Compose:** A tool for defining and running multi-container Docker applications. It uses a YAML file to configure the application's services (e.g., Flask app, PostgreSQL database), networks, and volumes, simplifying the entire application stack's deployment and management.

## 1.3. Database Schema & Models

The API exposes endpoints for managing the following SQLAlchemy models, providing a structured approach to food data:

- **User** : Manages user profiles. Includes fields such as `user_id` , `username` (unique), `email` (unique), `password_hash` (internal, hashed), `dietary_preferences` (e.g., Vegan, Omnivore), and `date_joined` .
- **FoodCategory** : Organizes food items into logical groups. Fields include `category_id` , `name` (unique), and `description` .
- **Food** : Represents individual food items with details like name, description, nutritional information (JSONB for flexible nutritional data), `category_id` (foreign key to `FoodCategory` ).
- **Ingredient** : Stores information about various cooking ingredients. Fields include `ingredient_id` , `name` (unique), and `description` .
- **FoodImage** : Manages image URLs associated with specific food items. Includes `image_id` , `food_id` (foreign key to `Food` ), `image_url` , and `description` .

(Note: Other models, such as `Allergen` , `WeeklyFoodPlanEntry` , `FoodIngredient` , `UserAllergy` , and `IngredientAllergen` , are defined in the project's SQLAlchemy models but do not have dedicated API endpoints implemented in this specific submission. They are placeholders for potential future API expansion.)

## 1.4. API Design Principles

The API follows RESTful design principles, aiming for:

- **Resource-Oriented**: Each major entity (User, Food, Ingredient) is treated as a resource accessible via a unique URL.
- **Statelessness**: Each request from client to server contains all necessary information.
- **Standard HTTP Methods**: Utilizes `GET` , `POST` , `PUT` , `DELETE` for standard CRUD operations.
- **JSON Communication**: Data is exchanged in JSON format for flexibility and broad compatibility.
- **Clear Status Codes**: Appropriate HTTP status codes (e.g., 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, 409 Conflict, 500 Internal Server Error) are used to indicate the outcome of API requests.

---

## 2. Prerequisites

To set up and run the Food App Backend API, ensure your environment meets the following specifications.

## 2.1. System Requirements

- **Operating System:** Windows 10/11 (Pro, Enterprise, Education), macOS (Intel or Apple Silicon), or Linux (e.g., Ubuntu, Debian, Fedora, CentOS).
- **RAM:** Minimum 4GB, 8GB or more recommended for smooth Docker operation.
- **Disk Space:** Minimum 20GB free space for Docker images, containers, and database data.
- **Internet Connection:** Required for initial Docker Desktop/Engine download and image pulls.

## 2.2. Software Installation & Verification

### 1. Docker Desktop (for Windows/macOS) / Docker Engine (for Linux):

#### ◦ Installation:

- **Windows/macOS:** Download the installer from the official Docker website: <https://www.docker.com/products/docker-desktop>. Follow the installation wizard. After installation, ensure Docker Desktop is running (you should see its icon in your system tray/menubar).
- **Linux:** Follow the specific installation instructions for your Linux distribution from the official Docker Engine documentation: <https://docs.docker.com/engine/install/>. Ensure you also install Docker Compose v2 (which is usually included with Docker Engine or installed separately as `docker-compose-plugin`).

- **Verification:** Open your preferred terminal or command prompt and execute the following commands. The output should confirm successful installation and version numbers.

```
docker --version
```

```
# Expected output example: Docker version 24.0.5, build 24.0.5-01
```

```
docker compose version
```

```
# Expected output example: Docker Compose version v2.20.2
```

```
# If your version is older (v1.x.x), ensure it's a v2 compatible
```

- **Important:** Ensure the Docker daemon is running before attempting to run any Docker commands.

---

## 3. Setup & Running the Application

This section provides a step-by-step guide to prepare and launch the Food App Backend API on your local machine using Docker Compose.

### 3.1. Obtaining the Project Files

## 1. Extract the Project Archive:

- Locate the provided submission file (e.g., `FOOD_APP_Submission.zip` or `FOOD_APP_Submission.tar.gz` ).
- Use your operating system's default extraction utility or a tool like 7-Zip (Windows), `tar` (Linux/macOS), or `unzip` (Linux/macOS) to extract the archive. This will create a main project folder (e.g., `FOOD_APP` ) containing all the application files.

## 2. Navigate to the Project Root Directory:

- Open your terminal or command prompt.
- Change your current working directory to the extracted `FOOD_APP` folder. This directory is the root of the project and contains key files such as `app.py` , `Dockerfile` , `docker-compose.yml` , `README.md` , and `.env.example` .

```
cd /path/to/your/extracted/FOOD_APP
# Example for Windows: cd C:\Users\YourUser\Projects\FOOD_APP
# Example for macOS/Linux: cd ~/Documents/Projects/FOOD_APP
```

## 3.2. Project Directory Structure

For clarity, here's a simplified overview of the critical files within the `FOOD_APP` directory relevant to running the application:

`FOOD_APP/`

- ├── `app.py` # Main Flask application file with API routes.
- ├── `Dockerfile` # Instructions for building the Flask app's Docker image.
- ├── `docker-compose.yml` # Defines the multi-container application services (app, db).
- ├── `requirements.txt` # Python dependencies for the Flask application.
- ├── `.env.example` # Template for environment variables (DATABASE\_URL, SECRET\_KEY).
- ├── `README.md` # This comprehensive guide.
- ├── `models/` # Contains SQLAlchemy ORM model definitions (e.g., `models.py` ).
  - | └── `models.py`
- ├── `database/` # Database configuration and ORM setup.
  - | └── `orm_config.py`
- ├── `migrations/` # (Generated by Flask-Migrate) Database migration scripts.
  - | └── `versions/`
- ├── `import_data.py` # (Optional) Script to seed initial database data.
- └── ... # Other project files and directories (e.g., `tests`, `business_logic`)

## 3.3. Environment Variable Configuration

The application relies on environment variables for sensitive configuration (like database credentials and Flask's secret key). These are loaded from a `.env` file, which is kept out of version control for security.

1. **Locate `** .env.example **`** : In the project root directory ( `FOOD_APP` ), find the file named `** .env.example **` .

2. **Create Your `.env` File**: In the same *directory* as `** .env.example **` , create a **new file** and name it `** .env **` .

- **Crucial**: Ensure the file is named exactly `.env` (without any file extension like `.txt` ).

3. **Populate Your `.env` File**:

- Open `** .env.example **` with a plain text editor (e.g., VS Code, Notepad++, Sublime Text, gedit, nano).
- Copy the entire content of `** .env.example **` .
- Paste this content into your newly created `** .env **` file.

4. **Customize the Values in `** .env **`** :

- **`DATABASE_URL`** : This specifies how your Flask application connects to the PostgreSQL database container.
  - **Recommended Value**:  
`postgresql://samuel:samuelDEV@db:5432/food_app_db_docker`
    - *Explanation*: `samuel:samuelDEV` are the default username and password for the PostgreSQL service as defined in `docker-compose.yml` . `db` is the hostname of the PostgreSQL service within the Docker Compose network. `5432` is the default PostgreSQL port, and `food_app_db_docker` is the database name.
    - (If you modified these values in your `docker-compose.yml` , update them here accordingly.)
- **`SECRET_KEY`** : This is a critical security key for Flask (used for session management, cookie signing, etc.). **You must replace `YOUR_FLASK_APP_SECRET_KEY_HERE` with a long, random, and unique string.**
  - **How to generate a strong `SECRET_KEY` (Python in terminal)**:

```
python -c "import os; print(os.urandom(24).hex())"
```

Copy the hexadecimal string output from this command and paste it as the value for `SECRET_KEY` in your `.env` file.

Example of a correctly configured `**.env**` file (for illustration, **do not copy these specific values directly**):

```
# .env
DATABASE_URL=postgresql://samuel:samuelDEV@db:5432/food_app_db_docker
SECRET_KEY=a5b6c7d8e9f01234567890abcdef1234567890abcdef1234567890abcc
```

5. Save the `**.env**` file.

## 3.4. Building & Starting Docker Containers

This step orchestrates the entire application stack: it builds the Docker image for your Flask application from your source code and `Dockerfile`, and then launches both your Flask application and the PostgreSQL database in interconnected Docker containers.

1. Ensure you are in the root of the project directory ( `FOOD_APP` ) in your terminal.
2. Execute the following command:

```
docker compose up --build -d
```

- `docker compose up` : This command reads the `docker-compose.yml` file and instructs Docker Compose to create and start the services defined within it ( `app` and `db` ).
- `--build` : This is a crucial flag. It forces Docker Compose to **rebuild the Docker image for your `app` service** (your Flask application) using the `Dockerfile` and the current source code within your project directory. This ensures that any changes to your Python code, `requirements.txt`, or `Dockerfile` itself are incorporated into the running container. It bypasses the Docker build cache for the `app` service, guaranteeing a fresh build.
- `-d` : This flag runs the containers in "detached" mode, meaning they will run in the background, and your terminal will remain free for other commands (e.g., checking logs, running migrations).

## 3.5. Verifying Container Status

After running `docker compose up --build -d`, it's important to confirm that both your Flask application and PostgreSQL database containers are running as expected.

Execute the following command in your terminal:

`docker ps`

You should see output similar to this, indicating that your containers are

| CONTAINER ID   | IMAGE                      | COMMAND          |
|----------------|----------------------------|------------------|
| <container_id> | food_app_backend_container | "flask run --hos |
| <container_id> | postgres:13-alpine         | "docker-entryp   |

NAMES: Confirms the expected container names.

STATUS: Should show Up X seconds (and ideally (healthy) if a healthcheck

PORTS: Shows that port 5000 on your host machine is mapped to port 5000 i

### 3.6. Database Initialization & Migrations

The very first time you set up the application with a new PostgreSQL data

Ensure your food\_app\_backend\_container is running (docker ps should confi

Execute the migration command inside the running Flask application contain

Bash

`docker compose exec app flask db upgrade`

`docker compose exec app:` This part of the command allows you to execute s

`flask db upgrade:` This is a command provided by Flask-Migrate (via Alembi

Expected Output: You should see informational messages from alembic indic

### 3.7. Populating Initial Data (Optional)

If your project includes an import\_data.py script (or similar) to pre-fil

Ensure your food\_app\_backend\_container is running.

Execute the data import script inside the Flask application container:

Bash

`docker compose exec app python import_data.py`

(Note: This step assumes your Dockerfile includes instructions to copy in

## 4. API Endpoints Reference

Once the application is successfully set up, running, and its database is

Base URL for all API Endpoints: `http://localhost:5000`

You can interact with these endpoints using any HTTP client tool, such as

Postman: A popular API platform for building and using APIs.

Insomnia: A free, open-source, and cross-platform API client.

curl: A command-line tool for making HTTP requests (useful for quick chec

For POST and PUT requests, ensure you set the Content-Type header to appl

### 4.1. User Management (/api/users)

This section details the API endpoints for managing User resources.

Model Fields: user\_id, username (unique), email (unique), password\_hash (



Retrieve All Users

Method: GET

URL: http://localhost:5000/api/users

Description: Fetches a list of all registered users in the system.

Response (200 OK):

JSON

```
[
  {
    "user_id": 1,
    "username": "john.doe",
    "email": "john.doe@example.com",
    "dietary_preferences": "Omnivore",
    "date_joined": "2024-01-15T10:30:00.000Z"
  },
  {
    "user_id": 2,
    "username": "jane.smith",
    "email": "jane.smith@example.com",
    "dietary_preferences": "Vegetarian",
    "date_joined": "2024-02-20T14:45:00.000Z"
  }
]
```

Create a New User

Method: POST

URL: http://localhost:5000/api/users

Description: Registers a new user account. username, email, and password

Request Body (JSON):

JSON

```
{
  "username": "newuser_example",
  "email": "newuser@example.com",
  "password": "strong_secure_password_123!",
  "dietary_preferences": "Pescatarian"
}
```

Required Fields: username (string, unique), email (string, unique, valid

Optional Fields: dietary\_preferences (string, allowed values: "Vegan", "V

Response (201 Created):

JSON

```
{
  "user_id": 3,
  "username": "newuser_example",
```

```
"email": "newuser@example.com",
"dietary_preferences": "Pescatarian",
"date_joined": "2024-06-12T15:00:00.000Z"
```

```
}
```

Error Responses (Examples):

```
400 Bad Request: {"message": "Missing username, email, or password"} (if
409 Conflict: {"message": "Username already exists"} (if username is not
409 Conflict: {"message": "Email already exists"} (if email is not unique
400 Bad Request: {"message": "'password': ['Password must be at least 6
400 Bad Request: {"message": "'dietary_preferences': ['Invalid dietary p
```

Retrieve User by ID

Method: GET

URL: `http://localhost:5000/api/users/<user_id>` (e.g., `http://localhost:5000/api/users/1`)

Description: Fetches detailed information for a single user identified by ID

Response (200 OK):

JSON

```
{
  "user_id": 1,
  "username": "john.doe",
  "email": "john.doe@example.com",
  "dietary_preferences": "Omnivore",
  "date_joined": "2024-01-15T10:30:00.000Z"
}
```

Error Response (404 Not Found):

JSON

```
{
  "message": "User with ID <user_id> not found."
}
```

Update User by ID

Method: PUT

URL: `http://localhost:5000/api/users/<user_id>` (e.g., `http://localhost:5000/api/users/1`)

Description: Updates existing user information. Accepts partial JSON data

Request Body (JSON - Example for partial update):

JSON

```
{
  "dietary_preferences": "Keto",
  "email": "john.updated@example.com"
}
```

Updatable Fields: username (string), email (string), dietary\_preferences (string)

(Note: This endpoint does not support changing the password directly for security reasons)

Response (200 OK):

JSON

```
{
  "message": "User updated successfully",
  "user": {
    "user_id": 1,
    "username": "john.doe",
    "email": "john.updated@example.com",
    "dietary_preferences": "Keto",
    "date_joined": "2024-01-15T10:30:00.000Z"
  }
}
```

Error Responses (Examples):

404 Not Found: {"message": "User with ID <user\_id> not found."}

409 Conflict: {"message": "Username already exists"} (if updating username)

409 Conflict: {"message": "Email already exists"} (if updating email to a

400 Bad Request: {"message": "{ 'email': ['Not a valid email address.'] }"}

Delete User by ID

Method: DELETE

URL: http://localhost:5000/api/users/<user\_id> (e.g., http://localhost:5000/api/users/1)

Description: Deletes a user record from the database.

Response (204 No Content): No response body; indicates successful deletion.

#### 4.2. Food Category Management (/api/categories)

This section details the API endpoints for managing FoodCategory resource.

Model Fields: category\_id, name (unique), description.

Retrieve All Food Categories

Method: GET

URL: http://localhost:5000/api/categories

Description: Fetches a list of all available food categories.

Response (200 OK):

JSON

```
[
  {
    "category_id": 1,
    "name": "Breakfast",
    "description": "Meals typically eaten in the morning."
  },
  {
    "category_id": 2,
    "name": "Dinner",
    "description": "Main meals for the evening."
  }
]
```

```

]
Create a New Food Category
Method: POST
URL: http://localhost:5000/api/categories
Description: Creates a new food category. name is mandatory.
Request Body (JSON):
JSON

{
    "name": "Snacks",
    "description": "Light bites between meals."
}
Required Fields: name (string, unique).
Optional Fields: description (string).
Response (201 Created):
JSON

{
    "category_id": 3,
    "name": "Snacks",
    "description": "Light bites between meals."
}
Error Responses (Examples):
400 Bad Request: {"message": "Missing category name"}
409 Conflict: {"message": "Category name already exists"}
Retrieve Food Category by ID
Method: GET
URL: http://localhost:5000/api/categories/<category_id> (e.g., http://loc
Description: Fetches detailed information for a single food category by i
Response (200 OK):
JSON

{
    "category_id": 1,
    "name": "Breakfast",
    "description": "Meals typically eaten in the morning."
}
Error Response (404 Not Found):
JSON

{
    "message": "Food category with ID <category_id> not found."
}
Update Food Category by ID

```

Method: PUT

URL: `http://localhost:5000/api/categories/<category_id>` (e.g., `http://localhost:5000/api/categories/1`)

Description: Updates an existing food category's information. Accepts parameters in the request body.

Request Body (JSON):

JSON

```
{
  "description": "Morning meals to start the day, often consumed before breakfast"
}
```

(You can update name or description.)

Response (200 OK):

JSON

```
{
  "message": "Category updated successfully",
  "category": {
    "category_id": 1,
    "name": "Breakfast",
    "description": "Morning meals to start the day, often consumed before breakfast"
  }
}
```

Error Responses (Examples):

404 Not Found: {"message": "Food category with ID <category\_id> not found"}

409 Conflict: {"message": "Category name already exists"} (if updating name)

Delete Food Category by ID

Method: DELETE

URL: `http://localhost:5000/api/categories/<category_id>` (e.g., `http://localhost:5000/api/categories/1`)

Description: Deletes a food category record.

Response (204 No Content): (No response body)

### 4.3. Food Item Management (/api/foods)

This section details the API endpoints for managing Food resources.

Model Fields: `food_id`, `name`, `description`, `nutritional_info` (JSONB), `category_id`

Retrieve All Food Items

Method: GET

URL: `http://localhost:5000/api/foods`

Description: Fetches a list of all available food items, including their category.

Response (200 OK):

JSON

```
[
  {
    "food_id": 101,
    "name": "Oatmeal with Berries",
    "description": "A bowl of oatmeal topped with fresh berries and a drizzle of honey.",
    "nutritional_info": {
      "calories": 350,
      "protein": 10,
      "carbs": 60,
      "fat": 5
    },
    "category_id": 1
  }
]
```

```

        "description": "Healthy breakfast with rolled oats and mixed berry",
        "nutritional_info": {"calories": 300, "protein_g": 10, "carbs_g": 40, "fat_g": 5},
        "category_id": 1,
        "category": {
            "category_id": 1,
            "name": "Breakfast"
        }
    }
]

```

Create a New Food Item

Method: POST

URL: <http://localhost:5000/api/foods>

Description: Creates a new food item. name and category\_id are mandatory.

Request Body (JSON):

JSON

```

{
    "name": "Grilled Salmon with Asparagus",
    "description": "Delicious and healthy grilled salmon fillet served with asparagus",
    "nutritional_info": {
        "calories": 450,
        "protein_g": 40,
        "fat_g": 25,
        "carbs_g": 10
    },
    "category_id": 2
}

```

Required Fields: name (string, unique), category\_id (integer, must exist)

Optional Fields: description (string), nutritional\_info (JSON object, flexible)

Dependency: The category\_id must correspond to an existing FoodCategory resource

Response (201 Created):

JSON

```

{
    "food_id": 102,
    "name": "Grilled Salmon with Asparagus",
    "description": "Delicious and healthy grilled salmon fillet served with asparagus",
    "nutritional_info": {"calories": 450, "protein_g": 40, "fat_g": 25, "carbs_g": 10},
    "category_id": 2,
    "category": {
        "category_id": 2,
        "name": "Dinner"
    }
}

```

Error Responses (Examples):

400 Bad Request: {"message": "Missing food name or category ID"}

404 Not Found: {"message": "Food category not found with the provided ID"}

409 Conflict: {"message": "Food with this name already exists"} (if name

Retrieve Food Item by ID

Method: GET

URL: http://localhost:5000/api/foods/<food\_id> (e.g., http://localhost:50

Description: Fetches detailed information for a single food item by its u

Response (200 OK):

JSON

```
{
  "food_id": 101,
  "name": "Oatmeal with Berries",
  "description": "Healthy breakfast with rolled oats and mixed berries.",
  "nutritional_info": {"calories": 300, "protein_g": 10, "carbs_g": 50}
  "category_id": 1,
  "category": {
    "category_id": 1,
    "name": "Breakfast"
  }
}
```

Error Response (404 Not Found):

JSON

```
{
  "message": "Food with ID <food_id> not found."
}
```

Update Food Item by ID

Method: PUT

URL: http://localhost:5000/api/foods/<food\_id> (e.g., http://localhost:50

Description: Updates an existing food item's information. Accepts partial

Request Body (JSON):

JSON

```
{
  "description": "Hearty breakfast with organic rolled oats, fresh mixe
  "nutritional_info": {
    "calories": 320,
    "protein_g": 12,
    "carbs_g": 55,
    "fiber_g": 8
  },
  "category_id": 1
}
```

```
}
Updatable Fields: name (string), description (string), nutritional_info (
(Note: If updating category_id, ensure it references an existing FoodCate
Response (200 OK):
JSON
```

```
{
  "message": "Food updated successfully",
  "food": {
    "food_id": 101,
    "name": "Oatmeal with Berries",
    "description": "Hearty breakfast with organic rolled oats, fresh
    "nutritional_info": {"calories": 320, "protein_g": 12, "carbs_g":
    "category_id": 1,
    "category": {
      "category_id": 1,
      "name": "Breakfast"
    }
  }
}
```

Error Responses (Examples):

```
404 Not Found: {"message": "Food with ID <food_id> not found."}
404 Not Found: {"message": "Food category not found with the provided ID"
409 Conflict: {"message": "Food with this name already exists"} (if updat
```

Delete Food Item by ID

Method: DELETE

URL: [http://localhost:5000/api/foods/<food\\_id>](http://localhost:5000/api/foods/<food_id>) (e.g., <http://localhost:5000/api/foods/101>)

Description: Deletes a food item record from the database.

Response (204 No Content): (No response body)

#### 4.4. Ingredient Management (/api/ingredients)

This section details the API endpoints for managing Ingredient resources.

Model Fields: ingredient\_id, name (unique), description.

Retrieve All Ingredients

Method: GET

URL: <http://localhost:5000/api/ingredients>

Description: Fetches a list of all available ingredients.

Response (200 OK):

JSON

```
[
  {
    "ingredient_id": 1,
    "name": "Eggs",
```



```

        "description": "Common protein source, versatile in cooking."
    }
]

Create a New Ingredient
Method: POST
URL: http://localhost:5000/api/ingredients
Description: Creates a new ingredient. name is mandatory.
Request Body (JSON):
JSON

{
    "name": "Spinach",
    "description": "Leafy green vegetable, rich in iron."
}

Required Fields: name (string, unique).
Optional Fields: description (string).
Response (201 Created):
JSON

{
    "ingredient_id": 2,
    "name": "Spinach",
    "description": "Leafy green vegetable, rich in iron."
}

Error Responses (Examples):
400 Bad Request: {"message": "Missing ingredient name"}
409 Conflict: {"message": "Ingredient with this name already exists"}

Retrieve Ingredient by ID
Method: GET
URL: http://localhost:5000/api/ingredients/<ingredient_id> (e.g., http://
Description: Fetches detailed information for a single ingredient by its
Response (200 OK):
JSON

{
    "ingredient_id": 1,
    "name": "Eggs",
    "description": "Common protein source, versatile in cooking."
}

Error Response (404 Not Found):
JSON

{
    "message": "Ingredient with ID <ingredient_id> not found."
}

```

```
}
Update Ingredient by ID
Method: PUT
URL: http://localhost:5000/api/ingredients/<ingredient_id> (e.g., http://
Description: Updates an existing ingredient's information. Accepts partial
Request Body (JSON):
JSON
```

```
{
  "description": "Cage-free organic eggs, excellent for baking and bread"
}
(You can update name or description.)
```

```
Response (200 OK):
JSON
```

```
{
  "message": "Ingredient updated successfully",
  "ingredient": {
    "ingredient_id": 1,
    "name": "Eggs",
    "description": "Cage-free organic eggs, excellent for baking and bread"
  }
}
```

Error Responses (Examples):

```
404 Not Found: {"message": "Ingredient with ID <ingredient_id> not found."}
409 Conflict: {"message": "Ingredient with this name already exists"} (if
Delete Ingredient by ID
```

```
Method: DELETE
URL: http://localhost:5000/api/ingredients/<ingredient_id> (e.g., http://
Description: Deletes an ingredient record.
Response (204 No Content): (No response body)
```

#### 4.5. Food Image Management (/api/food\_images)

This section details the API endpoints for managing FoodImage resources.

Model Fields: image\_id, food\_id (foreign key to Food), image\_url, description

Retrieve All Food Images

```
Method: GET
URL: http://localhost:5000/api/food_images
Description: Fetches a list of all stored food image records.
Response (200 OK):
JSON
```

```
[
  {
```

```
    "image_id": 1,
    "food_id": 101,
    "image_url": "[https://example.com/images/oatmeal_bowl.jpg] (https://example.com/images/oatmeal_bowl.jpg)"
    "description": "Overhead shot of oatmeal with berries."
  }
]
```

### Create a New Food Image

Method: POST

URL: [http://localhost:5000/api/food\\_images](http://localhost:5000/api/food_images)

Description: Creates a new food image entry. food\_id and image\_url are mandatory.

Request Body (JSON):

JSON

```
{
  "food_id": 101,
  "image_url": "[https://example.com/images/oatmeal_ingredients.jpg] (https://example.com/images/oatmeal_ingredients.jpg)"
  "description": "Ingredients for oatmeal preparation."
}
```

Required Fields: food\_id (integer, must exist in Food), image\_url (string)

Optional Fields: description (string).

Dependency: The food\_id must correspond to an existing Food item.

Response (201 Created):

JSON

```
{
  "image_id": 2,
  "food_id": 101,
  "image_url": "[https://example.com/images/oatmeal_ingredients.jpg] (https://example.com/images/oatmeal_ingredients.jpg)"
  "description": "Ingredients for oatmeal preparation."
}
```

### Error Responses (Examples):

400 Bad Request: {"message": "Missing food ID or image URL"}

404 Not Found: {"message": "Food item not found with the provided ID"} (if food\_id is not found)

400 Bad Request: {"message": "{ 'image\_url': ['Not a valid URL.'] }"} (if image\_url is not a valid URL)

### Retrieve Food Image by ID

Method: GET

URL: [http://localhost:5000/api/food\\_images/<image\\_id>](http://localhost:5000/api/food_images/<image_id>) (e.g., [http://localhost:5000/api/food\\_images/1](http://localhost:5000/api/food_images/1))

Description: Fetches detailed information for a single food image by its ID.

Response (200 OK):

JSON

```
{
  "image_id": 1,
  "food_id": 101,
  "image_url": "[https://example.com/images/oatmeal_bowl.jpg] (https://example.com/images/oatmeal_bowl.jpg)"
  "description": "Overhead shot of oatmeal with berries."
}
```

```

    "image_url": "[https://example.com/images/oatmeal_bowl.jpg] (https://e
    "description": "Overhead shot of oatmeal with berries."
}
Error Response (404 Not Found):
JSON

{
    "message": "Food Image with ID <image_id> not found."
}
Update Food Image by ID
Method: PUT
URL: http://localhost:5000/api/food_images/<image_id> (e.g., http://local
Description: Updates an existing food image's information. Accepts partial
Request Body (JSON):
JSON

{
    "image_url": "[https://example.com/images/oatmeal_serving.jpg] (https:
    "description": "Image of oatmeal served on a table."
}
Updatable Fields: food_id (integer), image_url (string), description (string)
(Note: If updating food_id, ensure it references an existing Food item.)
Response (200 OK):
JSON

{
    "message": "Food image updated successfully",
    "food_image": {
        "image_id": 1,
        "food_id": 101,
        "image_url": "[https://example.com/images/oatmeal_serving.jpg] (ht
        "description": "Image of oatmeal served on a table."
    }
}
Error Responses (Examples):
404 Not Found: {"message": "Food Image with ID <image_id> not found."}
404 Not Found: {"message": "Food item not found with the provided ID"} (if i
400 Bad Request: {"message": "{ 'image_url': ['Not a valid URL.'] }"} (if i
Delete Food Image by ID
Method: DELETE
URL: http://localhost:5000/api/food_images/<image_id> (e.g., http://local
Description: Deletes a food image record.
Response (204 No Content): (No response body)
5. Stopping the Application

```

To cleanly stop and remove the running Docker containers and their associ

Bash

```
docker compose down
```

Important Note: By default, docker compose down will also remove the db\_c

To stop containers but preserve your database data, use:

Bash

```
docker compose stop
```

You can then restart them later with `docker compose start`.

## 6. Troubleshooting Guide

If you encounter any issues while setting up or running the application,

Internal Server Error (HTTP 500) from API Endpoints:

This indicates an error occurred within the Flask application.

Solution: Check the Docker logs for your Flask app container. In your ter

RuntimeError: The current Flask app is not registered with this 'SQLAlche

This error typically occurs when Flask-SQLAlchemy loses its connection to

Solution: Ensure you have correctly rebuilt and restarted your Docker cor

Containers Failing to Start or Exiting Immediately:

If `docker ps` shows containers with Exited status, or they don't appear at

Solution: Check the comprehensive logs for the specific service that fail

Ensure that local ports 5000 (for your Flask app) and 5432 (for PostgresQ  
Database Table Not Found Errors (e.g., `psycopg2.errors.UndefinedTable`: re

This means your database tables haven't been created yet, or were reset.

Solution: You need to run the database migrations. Ensure your `food_app_k`

Bash

```
docker compose exec app flask db upgrade
```

If you previously ran `docker compose down` without preserving volumes (doc

API Validation Errors (e.g., `{"message": "Missing data for required field`

These are expected validation errors from your API, indicating that the i

Solution: Carefully review the "API Endpoints Reference" section for the

## 7. Contact Information

For any questions, clarifications, or issues regarding this submission, p

Your Full Name: TCHUENCHE KAMDEM SAMUEL YEDIDYA

Student ID: 23U2987

Email Address: yedidyakamdem@gmail.com

Thank you for reviewing my project.